Capstone Project Milestone 2: Working System and Preliminary Results

DataSage: An Agentic System for Data Quality, Cleaning, and Analysis · Group 25 · Repository: https://github.com/shrey-Bish/cse598-group25-datasage

Basic Information

Project title: DataSage: An Agentic System for Data Quality, Cleaning, and Analysis

Stakeholder name: Vyshanth Buddani **Meeting date:** October 19, 2026

Student name	Email	GitHub
Shrey Bishnoi	sbishno2@asu.edu	shrey-Bish
Arsha Jindal	ajinda20@asu.edu	Arshajindal
Ritik Agarwal	ragarw68@asu.edu	ritwiz06
Vatsal Nirmal	vnirmal3@asu.edu	vatsal-prog
Maulik Jadav	mjadav@asu.edu	maulik-jadav

Section 1 Stakeholder Follow Up and Final Deliverables

Meeting. October 19, 2026, [time], [Zoom or in person]. Participants: Vyshanth Buddani (stakeholder) and all five team members: Shrey Bishnoi, Arsha Jindal, Ritik Agarwal, Vatsal Nirmal and Maulik Jadav. **Notes:** [agenda, what the stakeholder said, decisions and action items].

What we presented. [What the team showed or discussed on October 19.]

What was useful and what is still missing. [Which results the stakeholder found useful, which needs remain unmet, and any changes he asked for.]

Table 1 lists the stakeholder's feedback, what we did about it, and what comes next. Rows 1 to 5 come from the Milestone 1 meeting on September 28, 2026. Rows 6 and 7 come from the October 19 meeting.

Table 1: Feedback → Team response → Next action.

#	Feedback	Team response	Next action
1	Correctness first: never change the original, log every change, ask before uncertain changes.	The original is kept read-only. Every fix creates a new version with a change-log entry (reason, rows and cells touched, risk, approval). Risk is set in code, so the model cannot talk its way past it.	Show the change log and replay script; ask if they are enough for him to trust an answer.
2	Only simple formatting fixes may run without asking.	Auto-applied: trimming spaces, merging labels that differ only by case or spaces, and parses that create no new missing values. Everything else asks. On his file DataSage asks 3 times.	Ask whether blanking one unreadable date should keep asking (now medium risk).
3	Stay focused on CSV/Excel and common tabular issues.	8 issue types, 7 check tools, 9 fix tools; no new formats were added.	Freeze the scope (below).
4	Start with a user rules file; trusted-document retrieval later.	`lookup_rules` reads a JSON rules file (allowed labels, ranges, formulas, date format). Retrieval is not built.	Keep retrieval optional.
5	Treat the numeric targets as initial goals; adjust after testing.	All targets were measured (Section 3). Accuracy and safety targets are close or met; speed with the large model is not.	Agree on the adjusted targets below.
6	[Milestone 2 feedback]	[Team response]	[Next action]
7	[Milestone 2 feedback]	[Team response]	[Next action]

Final deliverables and scope (our proposal; [confirm with the stakeholder at the meeting]):

- The DataSage command-line tool for one CSV or Excel table, a plain-English question and an optional rules file, covering the issue types in Section 2 and writing the five Milestone 1 outputs: quality report, change log, cleaned copy, answer with caveats, and a replay script.

- A simple Streamlit page for the demo, where the stakeholder uploads a file, answers the approval prompts and reads the report.
- Cleaning recipes: save an approved cleaning plan and replay it on next month's file with the same columns.
- A final evaluation on the held-out tasks plus 5 to 10 real messy public files, the stakeholder's ratings of 10 reports, and an acceptance test on files he chooses. Out of scope as before: joins across tables, unstructured data, machine learning, and writing back to the source.

Success criteria (proposed changes to Milestone 1 Table 4): keep answer accuracy ≥ 90% on held-out tasks, zero damaged valid cells or rows, median approvals ≤ 3 and tool-call success ≥ 95%. Report tokens per task instead of dollars, because the ASU endpoint we use has no per-call price. The 60-second target is the open question: only the 30B model meets it (Section 3), so we will ask the stakeholder to pick between a faster default model and a slower, larger one.

Section 2 Implemented Agentic System

DataSage is a single planner agent that calls tools in a loop. The model makes the judgment calls (what to check, which fix fits, when the data is clean enough), and plain Python does every computation. Figure 1 shows the implemented system. Solid boxes are built and evaluated, dashed boxes are planned, and NEW marks changes since Milestone 1.

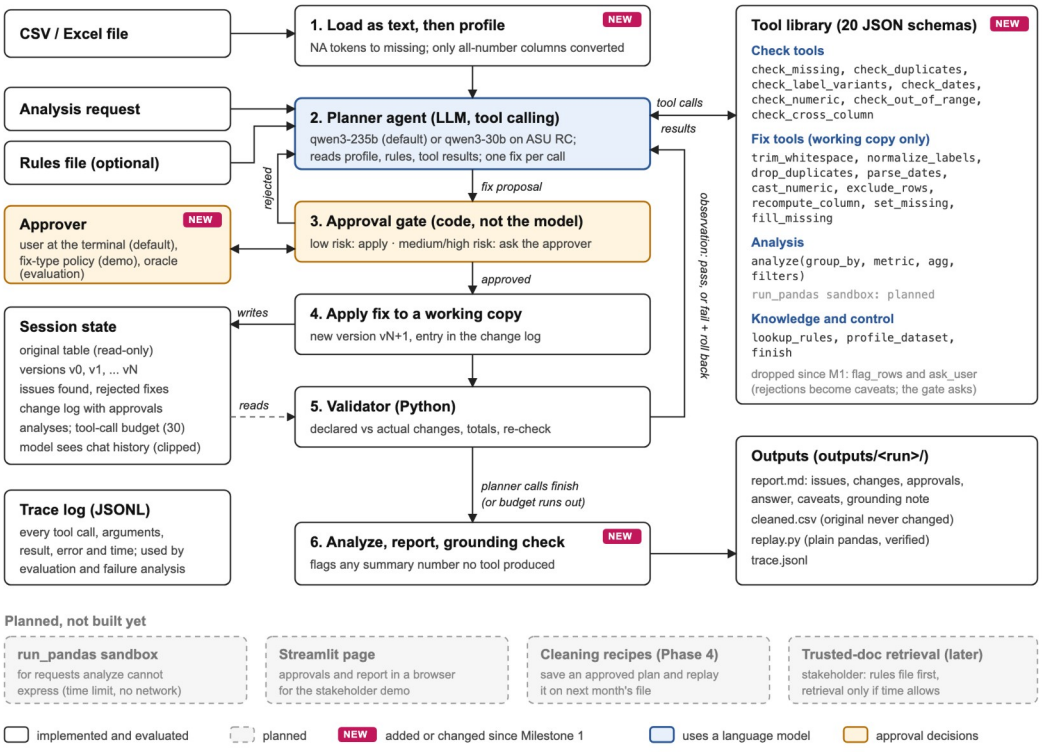


Figure 1: DataSage architecture as implemented for Milestone 2. Only the planner (blue) uses a language model. The approval gate and approver (orange) decide on risky fixes. Dashed boxes are not built yet. Full size: `docs/milestone2/architecture.png`.

2.1 Workflow from input to output

A user runs `python -m datasage --input file.csv --request "..." --rules rules.json`. Step numbers match Figure 1:

1. **Load and profile.** The file is read as text; blank, "NA" and "null" become missing, and a column becomes numeric only if every value is a plain number. The original is kept read-only as version v0.

2. **Plan.** The model gets the question, a profile of each column (type, missing count, distinct values, parse rates, case or space variants) and up to 12 sample rows, unusual rows first; it never sees the full table. It reads the rules and runs checks on the columns the question uses. Findings go into the issue list.
3. **Approval gate.** The model proposes one fix per call, with a reason. The fix runs as a dry run on a copy, and code sets its risk from the fix type and what changed. Low risk goes through; medium and high risk go to the approver with the table before and after. A rejected fix becomes a caveat and must not be retried.
4. **Apply.** An accepted fix becomes a new version (v1, v2, ...) with a change-log entry.
5. **Validate.** Before the commit, removed rows must match the declared rows, no other column may change, cell counts and totals must reconcile, and the original check is re-run. On failure the change is dropped and the problems go back to the model.
6. **Analyze, report, ground.** The model calls `analyze` (group by, metric, aggregation, filters), which refuses to add up a text column, then `finish`. The run writes `report.md`, `cleaned.csv`, `replay.py` (plain pandas that repeats the approved fixes) and `trace.jsonl`, and flags any summary number no tool produced.

Changes since Milestone 1. (a) The sandboxed `run_pandas` tool is not built yet, so analysis is structured only; every benchmark question fits that form. (b) We dropped `flag_rows` and `ask_user`: asking now happens inside each fix call through the gate, and a rejection becomes a caveat, so the model never decides on its own whether to ask. (c) The model sees the chat history with each tool result cut to 6,000 characters, not a compact state summary rebuilt every step. Runs are short (median 12 tool calls), so the history fits, but it costs tokens (Section 5). (d) After the failure analysis we added one rule to the system prompt about rule-breaking rows (Section 5); results are reported with and without it. (e) Dates are accepted only as YYYY-MM-DD, so a different format such as 03/04/2026 is flagged as invalid, not converted, and near-duplicate rows are not detected yet. Both were in the Milestone 1 scope and are listed in Section 6.

2.2 Agentic mechanisms

Table 2 lists each mechanism, when it runs, what it reads or updates, and how it changes what happens next.

Table 2: Agentic mechanisms in the implemented system.

Mechanism	When it is used	Reads / updates	Effect on the next action
Planner agent (one LLM with tool calling)	Every turn until `finish` or 30 tool calls	Reads the question, profile, sample rows and all earlier tool results	Picks the next check, fix, analysis or finish. Hitting the budget ends the run with a partial report.
Tools (20 JSON schemas: 7 checks, 9 fixes, `analyze`, `lookup_rules`, `profile_dataset`, `finish`)	Each call	Arguments validated with pydantic; bad calls return an error as text, for example the list of real column names	The model corrects the call instead of the run crashing. Outputs stay short (counts, a few row ids).
Session state (memory within a run)	Every check and fix	Read-only original, versions v0 to vN, issue list, rejected fixes, change log, analyses	Later checks and `analyze` run on the newest version. The report and replay script are built from the change log.
Rules file (knowledge)	`lookup_rules`, called first in every evaluated run	Allowed labels, ranges, formulas, date format	Supplies the ranges and formulas the model passes to `check_out_of_range` and `check_cross_column`, and the correct spellings for label fixes.
Approval gate with code-set risk tiers	Every fix	The dry-run result: rows removed, cells changed, new missing values	Rejected: not applied, becomes a caveat. Approved or low risk: goes to the validator.
Validator (verification during execution)	Every fix that passed the gate	Declared vs actual rows and cells, untouched columns, totals, re-run of the check	Fail: the change is discarded and the problems become the observation. Pass: new version.

Mechanism	When it is used	Reads / updates	Effect on the next action
Grounding check and trace	End of run; every call	Numbers in the summary vs tool outputs; each call, arguments, output and time	Unsupported numbers are listed as a warning. The trace drives scoring and `eval/failures.py`.

Why one agent. The work is sequential on one shared table, so a single planner avoids coordination cost. The validator and the risk tiers are plain code because row counts and totals must be checked exactly. The failures we found came from choosing the wrong fix, not from missing a second opinion, so we did not add the reviewer model mentioned in Milestone 1.

Models, frameworks and services. The planner is Qwen3-235B-A22B-Instruct-2507 by default, with Qwen3-30B-A3B-Instruct-2507 as the small model. Both are served by ASU Research Computing through an OpenAI-compatible API, called with the `openai` Python SDK at temperature 0. The rest is Python 3.13, pandas 3.0.5, pydantic 2, openpyxl and python-dotenv. We wrote the agent loop ourselves (`datasage/agent.py`, under 200 lines) instead of using an agent framework, so we control the state, the gate and the logging. Error handling: invalid arguments, unknown tools and validator failures go back to the model as text; API errors get the SDK's two retries with backoff and then end the run with a partial result. The repository has 13 unit tests (checks, fixes, risk tiers, validator, analysis, grounding), all passing.

Section 3 Preliminary Evaluation Results and Analysis

3.1 Datasets

We built a planted-error benchmark (`eval/generate.py`) so every task has an exact correct answer. The generator starts from a clean table, plants errors of the in-scope types and records each one (row id, column, type, messy value, clean value and the correct resolution: restore the value, exclude the row, set it to missing, keep it missing, or drop the duplicate copy). Applying those resolutions to the clean table gives a reference table, and each question's reference answer is computed on it with the same `analyze` code the agent uses. Each table gets two questions (counts, sums or means by group, some with a filter), chosen so they touch the columns with planted errors. Every table ships with a rules file. Table 3 summarizes the two datasets and the two splits.

Table 3: Datasets. Rows are per table; all systems ran on every task listed.

Dataset / split	Source	Tables	Rows	Tasks	Planted errors
Sales, dev	Synthetic orders with the same columns as the stakeholder's messy_sales.csv	10	41 to 65	20	96
Penguins, dev	Rows sampled from the public Palmer Penguins table (344 rows, CC0)	10	44 to 68	20	95
Sales, held-out	Same generator, new seed (101), made after the fix in Section 5	10	43 to 72	20	108
Penguins, held-out	Same generator, new seed (101)	10	43 to 67	20	88

What each one tests. Sales follows the stakeholder's file and use scenario: case variants and misspellings, duplicates, unreadable dates, text prices ("$1,200.00"), negative quantities and Revenue ≠ Quantity × Price. Penguins is a real public table from another domain (misspelled species and islands, impossible measurements, text numbers, duplicates), so DataSage is not tuned to one schema. **Splits:** the 40 dev tasks were used for all development and the failure analysis; the 40 held-out tasks were generated after the fix and run unchanged. **Limitations:** small tables, one generator for both splits, and no real messy files yet.

3.2 Baselines

Table 4: Baselines. None of them is DataSage.

Baseline	What it does	Why it is a fair comparison
B0: stakeholder baseline	Vyshanth's `run_baseline.py`, unchanged: five fixed checks, no cleaning. It has one built-in analysis, so we answer each question on the uncleaned file with our `analyze` code.	What a user gets today by trusting the raw file.

Baseline	What it does	Why it is a fair comparison
B1: fixed cleaning script	No model. Trims and merges case variants to the most common form, drops exact duplicates, converts mostly-numeric text, blanks invalid dates in columns named "date", drops rows with negative numbers. It is handed the question already structured.	Tests whether per-file decisions beat a careful fixed script. B1 gets the parsed question for free.
B2: single model call	The same 235B model gets the profile, sample rows, rules and question in one prompt and returns issues plus pandas code (`clean`, `answer`). We run the code once in a separate process (60 s limit); no checks, retries or approvals.	Tests whether the loop, tools, gate and validator add value over just asking the model.

3.3 Metrics and evaluation conditions

Table 5: Metrics, how we score them (`eval/scoring.py`) and the Milestone 1 targets.

Metric	Definition	M1 target
Answer correct	Same groups as the reference, every value within 0.5% (0.01 minimum)	≥ 90%
Completed (done)	An answer was produced within the budget	≥ 95%
Detection P / R	Reported (type, column) pairs vs planted ones. Milestone 1 said row level; checks report per column, so we score per column	≥ 0.85
Errors handled	Planted errors whose row ends in the right state (value restored, row removed, value blank, one copy left)	≥ 80%
Damage	Valid cells changed plus valid rows removed, compared with the reference table	0
Tool-call OK / recovery	Calls without an error; errors followed by a successful call	≥ 95% / ≥ 80%
Approvals	Approval requests per task (median)	≤ 3
Latency / tokens	Wall-clock seconds and prompt + completion tokens per task (median)	≤ 60 s / -

Conditions. All systems ran on the same tasks. Model calls use temperature 0. DataSage has a budget of 30 tool calls and 180 s per model call. Automated runs need someone to answer approvals, so an oracle built from the planted-error records (`eval/oracle.py`) stands in for the user: it approves a medium or high risk fix only if every removed row should be excluded or is a duplicate copy, and every changed cell gets its reference value. Anything else, such as filling a missing value, is rejected. B0, B1 and B2 never ask. DataSage and B2 must parse the question themselves; B0 and B1 get the structured form. Many tasks ran in parallel against a shared endpoint, so latency includes queueing. Each task ran once. No human or model-based grading is used; all scores are computed from the reference data.

3.4 Results

Table 6: Main results on the dev tasks. DataSage rows use the prompt without the exclusion rule from Section 5. Damage counts cells plus rows over all 20 tasks. Held-out accuracy for every system is in Table 7.

System	Answer correct	Done	Detect P / R	Errors handled	Damage	Tool OK	Approvals	Time (s)	Tokens
Sales, dev (20 tasks)									
B0 stakeholder baseline	0%	95%	0.80 / 0.61	19%	0	-	-	< 1	0
B1 fixed script	70%	100%	0.74 / 0.80	80%	0	-	-	< 1	0
B2 single model call (235B)	30%	80%	0.70 / 0.75	68%	195	-	-	23	2k
DataSage, 235B	85%	100%	0.84 / 0.85	89%	0	100%	3	152	56k
DataSage, 30B	65%	100%	0.78 / 0.76	75%	0	100%	3	16	55k
Penguins, dev (20 tasks)									
B0 stakeholder baseline	5%	80%	1.00 / 0.60	24%	0	-	-	< 1	0
B1 fixed script	45%	100%	1.00 / 0.81	80%	0	-	-	< 1	0
B2 single model call (235B)	45%	100%	0.86 / 0.74	79%	25	-	-	28	2k
DataSage, 235B	85%	100%	0.88 / 0.78	84%	0	100%	2	197	52k
DataSage, 30B	70%	100%	0.88 / 0.79	85%	0	100%	2	15	57k

Table 7: Answer accuracy on dev and held-out tasks (20 each), for the baselines and for DataSage before and after the exclusion rule (Section 5). The strict approver also rejects edits to rows that should be excluded.

System	Sales dev	Penguins dev	Sales held-out	Penguins held-out	Held-out damage
B0 stakeholder baseline	0%	5%	0%	5%	0
B1 fixed script	70%	45%	40%	30%	0
B2 single model call (235B)	30%	45%	85%	35%	31
DataSage, 235B	85%	85%	80%	45%	0
DataSage, 235B + exclusion rule	100%	100%	80%	95%	0
DataSage, 30B	65%	70%	95%	40%	0
DataSage, 30B + exclusion rule	95%	85%	90%	80%	0
DataSage, 30B + rule + strict approver	100%	85%	90%	65%	0

Table 8: Ablations (235B, first 10 dev tasks of each dataset, the same tasks for all three rows).

Dataset, system	Answer correct	Tasks	Damage
Sales, DataSage, 235B	100%	10	0
Sales, DataSage, all risky fixes rejected	30%	10	0
Sales, DataSage, no validator	100%	10	0
Penguins, DataSage, 235B	90%	10	0
Penguins, DataSage, all risky fixes rejected	0%	10	0
Penguins, DataSage, no validator	80%	10	0

3.5 Analysis

Where DataSage improves. On the dev tasks the 235B agent answers 85% of sales and 85% of penguin questions, against 70% and 45% for B1 and 30% and 45% for B2, with no damage. On the held-out tasks the same agent scores 80% and 45%: held-out penguins has the most rule-breaking rows, which trigger the main failure in Section 5. With the exclusion rule it reaches 80% and 95%, against 40% and 30% for B1 and 85% and 35% for B2. Over all 40 held-out tasks that is 88% against 35% for B1 and 60% for B2, which clears the Milestone 1 bar of a 15-point gain over both baselines but not the 90% accuracy target. It beats B1 because it reads the rules file: allowed labels let it fix most misspellings, such as "Electornics" and "Dram", and ranges let it drop impossible values such as a 4,750,000 g penguin, which B1 keeps because it only drops negatives. It beats B2 because B2 sees only 12 sample rows and cannot check its code: on held-out penguins 11 of its 13 wrong programs never filter impossible values; on sales dev 4 programs crashed, 11 of 14 wrong answers come from code that silently flips negative quantities with `abs()` (deciding the refund question without asking), and 5 tasks damaged valid data, for example by filling every missing Region with "Unknown".

Where it is similar or worse. The one place a baseline wins is held-out sales, where B2 scores 85%. In 3 of DataSage's 4 misses there, the check found misspellings such as "Furnture" and "Wst", but the model merged only the case variants and never proposed the spelling fix, which needs approval; 1 more ran out of its 30-call budget. B1 handles about 80% of planted errors with zero damage in under a second. DataSage's detection recall (0.85 and 0.78) is close to B1's and below target because it checks only the columns the question uses, a deliberate trade-off. Most of its false positives are a scoring artifact: with a minimum of 1 in the rules, `check_out_of_range` also reports negative rows as "out_of_range". Speed and size are the clear weak points: 152 to 197 s and about 56k tokens per task, against the 60 s target and about 2k tokens for B2.

Model size and the exclusion rule. The 30B model takes about 16 s but scored 65% and 70% on dev, with the same kind of mistakes as the 235B model. With the rule it reaches 95% and 85% on dev and 90% and 80% on held-out; on held-out tasks it fixed 10 and broke 3, with the biggest gain on penguins, which has the most rule-breaking rows. With the 235B model the rule raised dev accuracy from 85% and 85% to 100% and 100% (6 tasks fixed, 0 broken), but those are the tasks the rule was written on, so the held-out numbers are the fair test. The strict approver made no consistent difference: on 78 tasks where it rejected nothing, so approvals were identical, 6 still came out differently between the two runs, a measure of single-run noise.

Ablations and tool use (Table 8). Rejecting every risky fix lowers accuracy, as expected, but damaged nothing; across 20 runs the agent was told "no" 44 times and never retried a rejected fix. Turning the validator off changed only a few answers, within run-to-run noise, because the validator never failed a fix in any run: the fix tools are plain code and always did what they declared. It guards against bugs in those tools, which only the unit tests trigger. Over 474 tool calls the 235B model made 0 invalid calls (30B: 1 in 495), both read the rules first in every run, and approvals stayed at a median of 2 to 3 per task.

Not yet measured: real messy files, the stakeholder's ratings and acceptance test, cost in dollars and repeated runs (Section 6).

Section 4 Demo: Vyshanth's Sales File

Task and expected outcome. The input is the stakeholder's own `baseline/examples/messy_sales.csv` (8 rows) and the question "What was total revenue by category?", with a rules file (`examples/messy_sales_rules.json`) that lists the allowed categories and regions, says Quantity is at least 1, gives Revenue = Quantity × Price and the date format. His baseline reports Accessories $1,350 because a duplicate row is counted twice and "accessories" stays separate. In Milestone 1 we worked out the right answer by hand: Accessories $830, and Electronics $2,185 if the negative webcam row is a refund or $2,245 if it is an error. To act out a user who treats that row as a refund, we ran with `--approve tools --allow drop_duplicates,parse_dates`, which approves those two fix types and rejects everything else.

```
$ python -m datasage --input baseline/examples/messy_sales.csv --rules examples/messy_sales_rules.json \
    --request "What was total revenue by category?" --approve tools --allow drop_duplicates,parse_dates --out outputs/demo
[   4.7s] lookup_rules({}) -> ok
[   8.8s] check_duplicates({}) -> ok count=1 rows=[4]
[  31.0s] check_label_variants({"column": "Category"}) -> ok case/space={'electronics': 'Electronics', 'accessories': 'Accessories'} misspellings={}
[  33.9s] check_dates({"column": "Date"}) -> ok count=1 rows=[5]
[  36.8s] check_numeric({"column": "Quantity"}) -> ok text_numbers=0 negatives=1 not_numbers=0
[  45.1s] check_out_of_range({"column": "Quantity", "minimum": 1, "maximum": null}) -> ok count=1 rows=[6]
[  51.4s] check_cross_column({"column": "Revenue", "expression": "Quantity * Price"}) -> ok count=0 rows=[]
[  55.6s] check_missing({"column": "Region"}) -> ok count=1 rows=[7]
    [approval] drop_duplicates (medium risk): removed 1 exact duplicate row(s) -> APPROVED
[  66.6s] drop_duplicates({"reason": "One exact duplicate row found (row 4), which can skew revenue totals. Removing duplicate."}) -> ok removed 1 exact
duplicate row(s)
[  79.9s] normalize_labels({"column": "Category", "mapping": {"electronics": "Electronics", "accessories": "Accessories"}, "reason": "Sta) -> ok Category:
merged 2 label variant(s), 2 cells
    [approval] parse_dates (medium risk): Date: 1 invalid date(s) set to missing -> APPROVED
[ 117.7s] parse_dates({"column": "Date", "reason": "One invalid date 'not-a-date' (row 5) found. Setting invalid dates to missing to) -> ok Date: 1 invalid
date(s) set to missing
    [approval] exclude_rows (high risk): excluded 1 row(s): [6] -> REJECTED
[ 137.7s] exclude_rows({"rows": [6], "reason": "Row 6 has Quantity = -1, which violates the rule that Quantity must be at least 1. Th) -> ok The user
rejected this fix. Do not retry it; leave the data as is and mention it as a caveat.
[ 148.9s] analyze({"group_by": ["Category"], "metric": "Revenue", "agg": "sum"}) -> ok result={'Accessories': 830.0, 'Electronics': 2185.0}
[ 160.2s] finish({"summary": "Found and fixed: 1 duplicate row (removed), 2 label variants in Category (normalized to 'Electron) -> ok

stop: finish | tool calls: 14 | errors: 0 | approvals asked: 3 | 160.17s | tokens: 57081+566
answer: {'Accessories': 830.0, 'Electronics': 2185.0}
outputs written to outputs/demo/
```

Figure 2: Console output of the actual run. Each line is one tool call with its arguments and result. Indented lines are approval decisions (green approved, red rejected). The last lines give the stop reason, call count, approvals, time and tokens.

Walkthrough (Figure 2). The model reads the rules first, then runs one check per relevant column. `check_duplicates` finds row 4. `check_label_variants` finds "electronics" and "accessories" as case variants. `check_dates` finds the unreadable date in row 5. `check_numeric` and `check_out_of_range` (with the rules' minimum of 1) both point at row 6, Quantity −1. `check_cross_column` with the rules' formula finds no Revenue mismatch, so no recompute is proposed. `check_missing` finds the blank Region in row 7, which the question does not use, so it is left alone and reported. Then it fixes the problems one at a time:

- `drop_duplicates` removes a row, so the gate rates it medium risk and asks. Approved, and the validator confirms that exactly the declared row was removed: version v1.
- `normalize_labels` merges two labels that differ only by case. Low risk, applied with no question: v2.
- `parse_dates` turns one value into a missing value, which raises it to medium risk. Approved: v3.
- `exclude_rows` on row 6 is high risk. The approver rejects it, the tool answers "Do not retry it; leave the data as is and mention it as a caveat", and the model moves straight to `analyze`.

outputs/demo/report.md (rendered), sections: Changes, Answer, Summary

Changes

Version	Fix	Risk	Approval	What changed	Reason
v1	drop_duplicates	medium	approved	removed 1 exact duplicate row(s)	One exact duplicate row found (row 4), which can skew revenue totals. Removing duplicate.
v2	normalize_labels	low	auto	Category: merged 2 label variant(s), 2 cells	Standardizing category labels to match allowed values in rules. Variants differ only by case, so low risk.
v3	parse_dates	medium	approved	Date: 1 invalid date(s) set to missing	One invalid date 'not-a-date' (row 5) found. Setting invalid dates to missing to comply with rules. This row will still be included in revenue by category as Date is not used in the question.
-	exclude_rows	high	rejected	excluded 1 row(s): [6] (not applied)	Row 6 has Quantity = -1, which violates the rule that Quantity must be at least 1. This is likely a return or error. Excluding to prevent negative revenue from skewing totals.

Answer

sum of Revenue by ['Category'] (rows used: 7, version v3)

Group	Value
Accessories	830.00
Electronics	2,185.00

Summary

Found and fixed: 1 duplicate row (removed), 2 label variants in Category (normalized to 'Electronics' and 'Accessories'), and 1 invalid date (set to missing). The row with negative Quantity (-1) was not excluded due to user rejection, so the total revenue includes a negative contribution of -60 from that row. All other data issues were resolved. Total revenue by category: Accessories: 830.0, Electronics: 2185.0.

Figure 3: Part of the report.md written by the same run: the change log with the risk and approval of each fix (including the rejected one), the answer table and the model's summary with the caveat. The full report also lists every problem found (`docs/milestone2/demo/report.md`).

Result. `analyze` runs on v3 and returns Accessories 830 and Electronics 2,185, the expected answer for the refund reading. The summary states that the −1 row was kept because the user rejected excluding it. The run used 14 tool calls with no errors, asked 3 questions (the Milestone 1 limit) and took 160 s, well over the 60 s target. We re-ran the generated `replay.py` without the agent and it produced exactly the same 7 rows and 56 cells as `cleaned.csv`.

Link to the architecture. The run shows every implemented box in Figure 1: planning with the rules file (steps 1 and 2), the gate setting risk in code and the approver deciding (step 3), versions and the change log in the session state (step 4), the validator passing each applied fix (step 5), and the report, replay script and caveat (step 6). It also shows why the task needs an agent: whether the −1 row stays depends on the user's answer, and the model adapted its plan to the rejection instead of repeating the fix.

Section 5 Failure Analysis and Current Limitations

`eval/failures.py` reads the trace and planted errors of every wrong task and lists likely causes (for example "never ran a check for out_of_range" or "checked another column but not the one with the error"). We read the traces of the main cases by hand to confirm the label.

Case 1: rule-breaking rows blanked instead of removed (main failure)

```
$ python -m eval.run_eval --system datasage --dataset sales   # task sales_05_q0, trace excerpt
  question: "How many orders are there in each category?"   reference: Electronics 13, Furniture 11
[  38.6s] check_numeric({"column": "Quantity"})
          -> negatives={'count': 2, 'rows': [28, 50]}
[  43.9s] check_out_of_range({"column": "Quantity", "minimum": 1, "maximum": null})
          -> count=2 rows=[28, 50]
[ 110.6s] set_missing({"column": "Quantity", "rows": [28, 50], "reason": "Quantity values are negative (-8 and -4), but the rules state Quantity must be at
least 1. These a..."})
          -> Quantity: set 2 value(s) to missing  [risk=high, approval=approved]
[ 136.7s] analyze({"group_by": ["Category"], "metric": "Order_ID", "agg": "count"})
          -> result={'Accessories': 20.0, 'Electronics': 14.0, 'Furniture': 12.0, 'Office Supplies': 11.0}
score: analysis_correct=False   Electronics 14 (ref 13), Furniture 12 (ref 11): rows 28 and 50 still counted
```

Figure 4: Trace excerpt from task sales_05_q0 (DataSage, 235B, dev). Both checks point at rows 28 and 50; the planner blanks Quantity in those rows instead of excluding them, so the count still includes them.

Where it occurred: in planning, when choosing the fix, and it got past our approval oracle. **Observed (Figure 4):** the question is "How many orders are there in each category?". `check_numeric` and `check_out_of_range` both report rows 28 and 50, whose Quantity (−8 and −4) breaks the rule "Quantity must be at least 1". The planner calls `set_missing` on those two cells instead of `exclude_rows`. This is high risk, so the oracle is asked, and it approves: it only rejected edits to rows that should be kept, and these rows were going to be excluded anyway. The validator passes because the fix did exactly what it declared. `analyze` then counts rows per category and the two rows still count: Electronics 14 instead of 13 and Furniture 12 instead of 11. **Evidence across the benchmark:** this pattern is the first-listed cause for 6 of the 6 wrong dev tasks for the 235B model and 11 of 13 for the 30B model. It only hurts questions that count rows or use another column of the bad row; a sum of Quantity skips the blank anyway. **Suspected cause (not proven):** the prompt asks for "the smallest correct fix" and says not to change values it is unsure about, which pushes the model toward a one-cell edit.

Fix: one rule added to the system prompt (`EXCLUSION_RULE` in `datasage/agent.py`): when a row breaks a rule, propose `exclude_rows` and let the user reject it; use `set_missing` only for unreadable values in rows that are otherwise valid. We also tried a stricter approver that rejects any edit to such rows, like a user whose policy is "drop rows that break a rule". **How we checked it:** we re-ran all 40 dev tasks, and because the fix was found on dev tasks we also ran it on the 40 held-out tasks generated afterwards (Table 7). With the 30B model the rule fixed 10 dev tasks and broke 1 (penguins_00_q0), and the blanking pattern went from 11 failures to 0. With the 235B model it fixed 6 dev tasks and broke 0.

Case 2: partial success, the metric column was never checked

Where it occurred: planning (which columns to check). **Observed:** in task penguins_00_q0 ("What is the average body mass for each sex?") the 30B model with the new rule did several things right. It merged " Adelie" and "Chinstrap ", fixed "Bisco" and "Dram", removed a duplicate, and correctly excluded the two rows with negative flipper lengths. But it never ran a check on `body_mass_g`, the column it was averaging. Two planted values of 4,750,000 g and 3,550,000 g (grams typed as milligrams) stayed in, and the female average came out as 254,991 g instead of 3,700 g. The answer is obviously wrong to a person, but nothing in the run looked at it. **Evidence:** the trace lists every call, and no check names `body_mass_g`; `eval/failures.py` gives the same cause for 2 of the 4 remaining 30B failures after the fix. **Proposed fix:** a coverage check in code before `analyze` runs: every column the call uses (group by, metric, filters) must have been checked at least once, with a range check when the rules give a range; otherwise `analyze` returns an error naming the column, so the model checks it first. **How we will check it:** re-run the failed tasks and the full held-out set; success means no "never checked" failures and no lower accuracy elsewhere.

Broader limitations

- **Speed and token use.** The 235B agent needs a median of 152 s and about 56k tokens per sales task; the 30B model takes 16 s. Most tokens come from resending the whole conversation to the model on every turn of a run (a median of about 12 tool calls). A compact state summary (Section 6, item 3) should cut this.
- **Coverage and data.** Only group-by, aggregate and filter questions on one table, dates only as YYYY-MM-DD, and no near-duplicate check. All errors are planted by our own generator into small tables, so real messy files (Section 6, item 4) are needed before we can say the numbers carry over.
- **Simulated user.** The oracle knows the right answer. A real user may approve a wrong fix or reject a right one. The reject-all ablation covers one extreme; the stakeholder's ratings will show real behavior.
- **Verification gaps.** The validator never failed a fix in any evaluated run, so only unit tests show that it catches a bad one. It checks that a fix did what it declared, not that it was the right fix (Case 1). The grounding check only warns and does not block a report.
- **Single runs on a shared service.** Temperature 0 does not make runs repeat exactly, so a difference of one or two tasks is within noise until we repeat runs. If the ASU endpoint is slow or down, DataSage stops with a partial result.

Section 6 Remaining Work Before the Final Submission

Table 9 lists the remaining work in priority order. Items 1 to 8 are required; item 9, the `run_pandas` sandbox and trusted-document retrieval are optional. Owners are [to be agreed by the team].

Table 9: Remaining work.

#	Item	Current gap	Planned work or experiment	Owner	Done when
1	Stakeholder review and scope sign-off	Oct 19 meeting was before the system was built	Send the demo and results for his review (input form); record his answers in Section 1	[owner]	Scope, deliverables and targets confirmed in writing
2	Coverage check and spelling merges	Metric column sometimes never checked (Section 5, case 2); misspellings found but not merged (held-out sales)	Code check in `analyze`; prompt rule to propose merges into labels the rules allow; re-run held-out tasks	[owner]	No "never checked" or unmerged-misspelling failures on held-out, no accuracy drop
3	Default model and speed	235B median well over 60 s; ~50k tokens per task	Send a compact state summary instead of full history; compare 235B and 30B with the rule on held-out	[owner]	Stakeholder picks a model based on measured accuracy vs time
4	Real messy files	Only planted errors so far	5 to 10 public files with real problems, labeled by two members	[owners]	Labeled set and results for all systems
5	Repeated runs	One run per task	3 runs of DataSage and B2 on held-out tasks	[owner]	Mean and range reported
6	Stakeholder ratings and acceptance test	Not started	He rates 10 reports 1 to 5 (correctness, clarity, trust) and runs files he chooses	[owner]	Ratings and pass or fail in the final report
7	Streamlit demo page and cleaning recipes	Command line only; no recipes	Upload, approve and read the report in a browser; save an approved plan and replay it on a new file	[owners]	Stakeholder uses the page at Demo Day; a recipe replays with the validator passing
8	Final report, presentation and demo	-	Write up final results and failure cases	All	Submitted
9	Date formats and near-duplicates	Only YYYY-MM-DD; no near-duplicate check	Convert unambiguous formats; flag ambiguous ones like 03/04 for approval	[owner]	Planted-error tests for both pass

Section 7 GitHub Repository and Team Contributions

Repository: https://github.com/shrey-Bish/cse598-group25-datasage (public). **Branch:** `main`. **Commit for this submission:** `0b06910`. The README gives setup (Python 3.11+, `pip install -r requirements.txt`, an `.env` with the ASU endpoint key), the demo command from Section 4, and the commands that regenerate the benchmark and every table in Section 3. All per-task results and traces are in `eval/results/`; the demo outputs are in `docs/milestone2/demo/`.

Table 10: Team contributions.

Member (GitHub)	Contributions	Commits / PRs
Shrey Bishnoi (shrey-Bish)	Agent, tools, approval gate, validator, analysis and report builder (`datasage/`); baselines B1 and B2; benchmark generator, scoring, oracle, failure classifier and held-out split; tests; evaluation runs; diagram, demo, report draft.	0246a25, 0c83405, e54a845, ffa7f5a, a0a7003, 5717203, 52ccaa1, 0b06910
Arsha Jindal (Arshajindal)	[Contributions]	[Links]
Ritik Agarwal (ritwiz06)	[Contributions]	[Links]
Vatsal Nirmal (vatsal-prog)	[Contributions]	[Links]
Maulik Jadav (maulik-jadav)	[Contributions]	[Links]
Work not shown in commits	[Stakeholder communication, meeting notes, reviewing results, report writing: who did what]	-